



Scalable and Reliable Services (A.A. 2025/26)

Il corso affronta le sfide ingegneristiche legate alla creazione di sistemi distribuiti moderni, dove la complessità deriva dalla necessità di gestire enormi volumi di dati e garantire il funzionamento anche in caso di guasti parziali.

Indice

- [Scalable and Reliable Services \(A.A. 2025/26\)](#)
 - [Indice](#)
 - [1. I Pilastri del Corso: Scalabilità e Affidabilità](#)
 - [2. Architetture Cloud-Native e Microservizi](#)
 - [3. Teoria dei Sistemi Distribuiti](#)
 - [4. Laboratorio e Progetto Finale](#)
 - [5. Risorse e Supporto](#)
-

1. I Pilastri del Corso: Scalabilità e Affidabilità

Il "contratto" didattico si fonda su due concetti che spesso sono in contrasto tra loro:

- **Scalabilità:** Non si tratta solo di "reggere" più utenti, ma della capacità del sistema di mantenere le prestazioni costanti all'aumentare del carico, aggiungendo risorse (nodi, memoria, CPU). Si analizzerà la differenza tra scalabilità verticale (più potenza su una macchina) e orizzontale (più macchine in parallelo).
 - **Affidabilità (Reliability):** Un sistema affidabile deve continuare a erogare il servizio correttamente anche se alcuni componenti falliscono. Questo implica lo studio di tecniche di ridondanza, monitoraggio e self-healing.
-

2. Architetture Cloud-Native e Microservizi

Il corso abbandona il concetto di monolito per concentrarsi su architetture distribuite:

- **Microservizi:** Scomposizione di un'applicazione in servizi indipendenti che comunicano tramite rete (spesso via gRPC o REST). Questo introduce problemi di rete, latenza e consistenza dei dati.
 - **Orchestrazione:** L'utilizzo di Docker per la containerizzazione e Kubernetes per la gestione automatizzata dei carichi di lavoro su cluster di macchine.
-

3. Teoria dei Sistemi Distribuiti

Per progettare servizi scalabili, bisogna comprendere i limiti fisici e matematici della distribuzione:

- **Teorema CAP:** La scelta forzata tra Consistenza, Disponibilità e Tolleranza alle partizioni di rete.
 - **Consenso Distribuito:** Come far sì che più macchine si accordino su un unico valore (fondamentale per database distribuiti e sistemi di configurazione).
 - **Modelli di Guasto:** Capire come un sistema può fallire (crash, omissioni, errori bizantini) per progettare contromisure adeguate.
-

4. Laboratorio e Progetto Finale

La componente pratica è centrale e richiede di "sporcarsi le mani" con le tecnologie di settore:

- **Sviluppo Hands-on:** Implementazione di servizi che devono dimostrare tolleranza ai guasti (fault tolerance) e bilanciamento del carico (load balancing).
 - **Esame:** Consiste nella discussione di un progetto originale. Non basta che il codice funzioni; deve essere dimostrata la resilienza dell'architettura attraverso test di carico o simulazioni di guasto.
-

5. Risorse e Supporto

Il materiale comprende slide aggiornate, paper scientifici classici (come quelli di Google o Amazon sulle basi dei sistemi distribuiti) e documentazione tecnica. Il docente è disponibile per revisioni dell'architettura del progetto durante le ore di ricevimento.